Checkpoint-Aware GPU Job Scheduler

A C++ and PyTorch prototype for serving latency-sensitive inference jobs during long training runs

Metric	FCFS	Checkpoint
Median average short-job wait	8.82 s	4.11 s
Median training completion	5.16 s	14.27 s
Median wait reduction	-	52.2%
Median additional training time	-	9.10 s

Prepared by Karan Ragireddy

Five-run controlled benchmark | November 2026

1. Problem and objective

A shared GPU can be occupied by a long training job while a short inference request waits. The project evaluates whether cooperative checkpointing can release the device, serve queued work, and resume training without changing the final model state.

2. System design

The C++ scheduler launches and observes Linux processes using fork, exec, and waitpid. A PyTorch worker checks for a pause request between optimizer steps. It saves the completed step, model weights, and optimizer state to a temporary file, then atomically publishes the checkpoint and exits. The scheduler serves queued inference work and launches a fresh worker in resume mode.

Execution sequence

1	**Train**	Long-running worker owns the device.
2	**Request**	Short job enters the queue.
3	**Checkpoint**	Training state is atomically committed.
4	**Serve**	Trainer exits and inference runs.
5	**Resume**	Training restarts from the committed step.

3. Correctness verification

The correctness notebook compares a resumed training run with an uninterrupted reference and checks every final parameter using PyTorch tolerance. The queue benchmark also confirms that both short jobs execute under each policy. The submitted run produced the expected PASS result.

4. Benchmark results

Each of five runs submitted two inference jobs during a paced 100-step training workload. The table reports medians across the five runs.

Metric	FCFS	Checkpoint
Median average short-job wait	8.82 s	4.11 s
Median training completion	5.16 s	14.27 s
Median wait reduction	-	52.2%
Median additional training time	-	9.10 s

Interpretation

Checkpoint scheduling reduced the median average wait from 8.82 seconds to 4.11 seconds. Both queued jobs started sooner in every recorded run. The median 9.10-second increase in training completion includes checkpoint writing, worker termination, inference-process startup, inference execution, and training restart.

5. Limitations

The experiment uses a synthetic model, controlled sleep-based pacing, a filesystem queue, and cooperative workers. The timings describe this prototype only. The current logs do not separate checkpoint cost, GPU release, inference startup, inference execution, and restart cost. The benchmark policy order is fixed in the supplied notebook.

6. Next engineering milestones

Add phase-level timestamps, alternate policy order, evaluate real training and inference workloads, add priority aging, verify GPU-memory release, and replace queue files with a Unix-domain socket interface. Multi-GPU placement and crash recovery would complete the transition from a demonstrator to a reusable scheduler.

7. Reproduction

Upload the two notebooks from the repository to Google Colab. Run the correctness notebook first, then the queue benchmark. The repository also contains the C++ source, PyTorch worker, five raw run files, result summarizer, tests, and browser-based GitHub upload guide.